Pakistan AQI Predictor

A Serverless, End-to-End Air Quality Forecasting System

10Pearls Internship Project Report

Submitted by

Muhammad Arbaz Khalid

BSDS, 7th Semester — Riphah International University

Internship Duration: July 13 – September 4, 2026

Supervised by

Hafsa & Ummama

September 2026

1. Introduction 3
1.1 Objectives 3
2. System Architecture 4
2.1 Components 5
3. Data Pipeline 6
3.1 Data Collection 6
3.2 Ground-Truth Validation 6
3.3 AQI Calculation 6
3.4 Storage 6
4. Calibration & Bias Correction 6
5. Model Training & Comparison 8
5.1 Models Compared 8
5.2 Results — 24 Hour Horizon 8
5.3 Results — 48 Hour Horizon 9
5.4 Results — 72 Hour Horizon 9
5.5 Explainability 10
6. Automation & CI/CD 12
6.1 Hourly Feature Pipeline 12
6.2 Automated Retraining Pipeline 13
6.3 Automated Commit Evidence 14
7. Dashboard 14
7.1 Home 14
7.2 Compare Cities 16
7.3 Health Advice 17
7.4 Model Performance 17
8. Challenges & Learnings 17
9. Tech Stack Deep-Dive 18
10. Known Limitations 19
11. Conclusion 20
12. References & Links 20

1. Introduction

Air pollution is a persistent and severe public health issue across Pakistan, with several major cities regularly ranking among the most polluted in the world, particularly during the winter smog season. Despite this, access to reliable, city-level air quality data — and especially forward-looking forecasts — remains limited for much of the country, with real-time ground monitoring stations installed in only a handful of major cities.

This project, developed as part of a 10Pearls internship, addresses that gap by building a serverless system that estimates and forecasts Air Quality Index (AQI) for 30 Pakistani cities, using satellite/model-based pollutant data where physical monitoring stations are unavailable, and validating those estimates against real ground-truth readings where they exist. Beyond estimating current conditions, the system forecasts AQI up to three days ahead using machine learning models trained on historical pollutant and weather patterns and presents the results through a public, interactive dashboard.

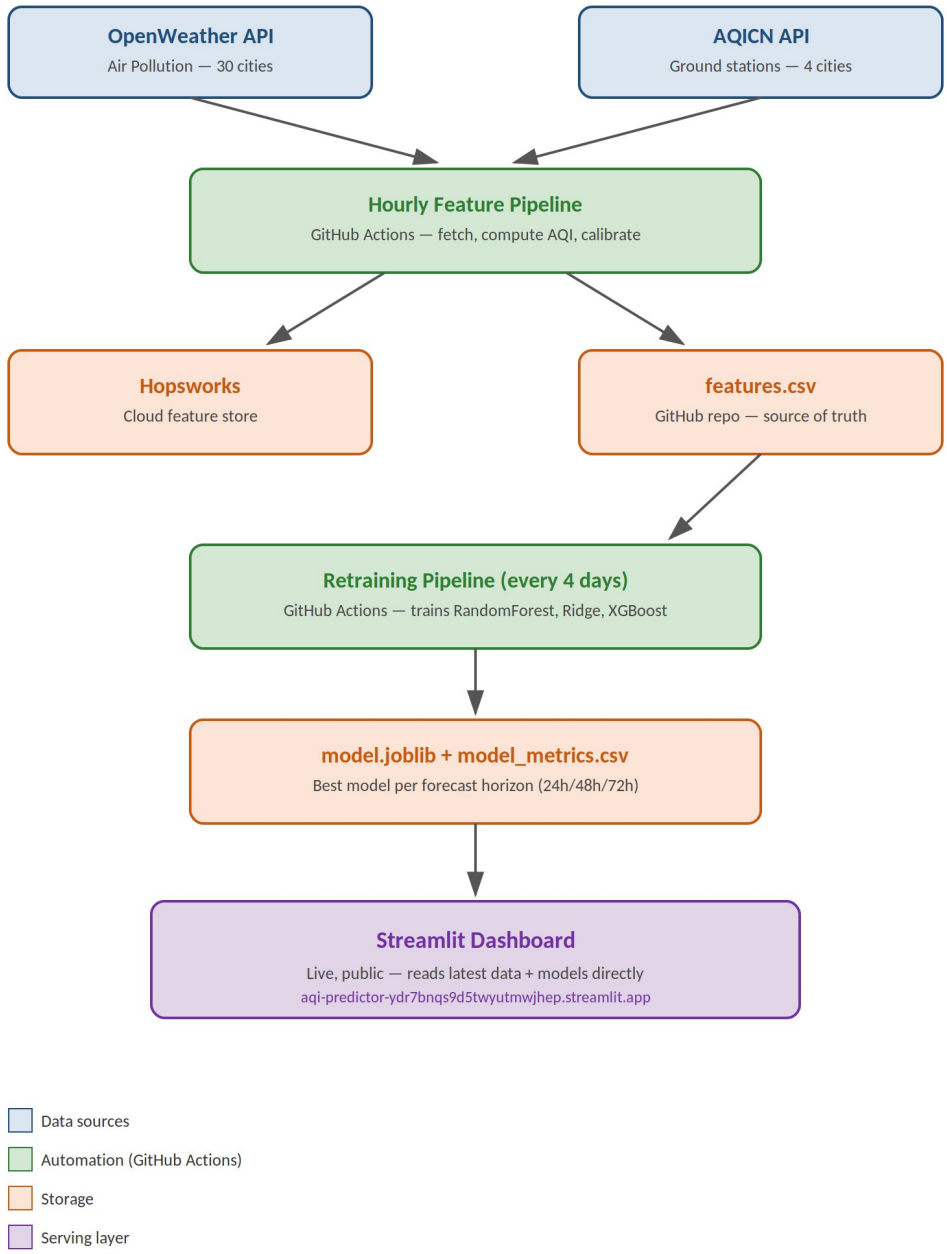
This report documents the system's architecture, data pipeline, model training and evaluation process, calibration methodology, automation approach and the resulting dashboard — along with the limitations encountered and lessons learned throughout development.

1.1 Objectives

- Collect live air quality and weather data for 30 Pakistani cities on an hourly basis.
- Compute standardized AQI using the official US EPA methodology.
- Validate and calibrate model estimates against real ground-truth monitoring stations.
- Train and compare multiple machine learning models to forecast AQI up to 3 days ahead.
- Automate the entire pipeline — data collection, model retraining and deployment — with no manual intervention.
- Present the results through an interactive, publicly accessible dashboard.

2. System Architecture

The system is built entirely on serverless infrastructure — there are no servers to provision, patch, or maintain. Every component runs on managed or scheduled cloud services: data collection and model retraining run as scheduled GitHub Actions workflows, storage is handled by a managed feature store and version-controlled files, and the dashboard is served by Streamlit Cloud. Data flows in one direction through the system: external APIs feed an hourly collection pipeline, which writes to storage; a separate retraining pipeline periodically reads that stored data to produce updated models; and the dashboard reads the latest data and models directly, with no manual step connecting any of these stages.



2.1 Components

- **Data sources:** OpenWeather Air Pollution API (all 30 cities) and AQICN ground-station API (4 validation cities).
- **Feature pipeline:** hourly GitHub Actions workflow that fetches, computes AQI, and stores data.
- **Feature store:** Hopsworks (cloud feature store) plus a version-controlled local CSV as the source of truth.
- **Training pipeline:** a second GitHub Actions workflow, scheduled every 4 days, that retrains all models.
- **Serving layer:** a Streamlit dashboard reading directly from the repository's latest data and model files.

3. Data Pipeline

3.1 Data Collection

Weather and pollutant concentration data is collected every hour for 30 Pakistani cities via GitHub Actions, using OpenWeather's Air Pollution API as the primary source. This API is atmospheric-model-based and works for any latitude/longitude, which is what makes nationwide coverage across all 30 cities possible without needing a physical sensor in each one.

3.2 Ground-Truth Validation

In parallel, real ground-station readings are pulled from the AQICN network for 4 major cities — Karachi, Lahore, Islamabad, and Peshawar — where physical monitoring stations exist. These readings serve as ground truth to evaluate how accurate the model-based estimates are.

3.3 AQI Calculation

Rather than relying on a third-party AQI figure, AQI is computed directly from PM2.5 and PM10 concentrations using the official US EPA breakpoint formula, including proper value truncation as specified by EPA methodology. This ensures the AQI values in this project are standardized and independently verifiable.

3.4 Storage

Feature data is written to two destinations: a Hopsworks feature store (for structured, queryable feature management) and a local CSV file tracked in version control, which serves as the pipeline's primary source of truth and what the dashboard and training pipeline both read from.

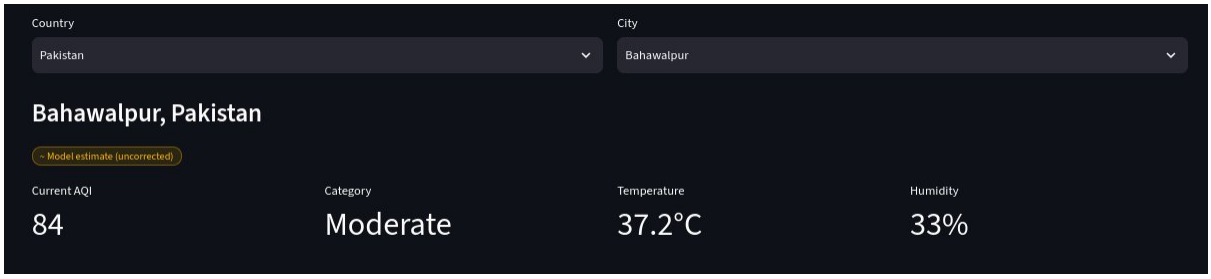
4. Calibration & Bias Correction

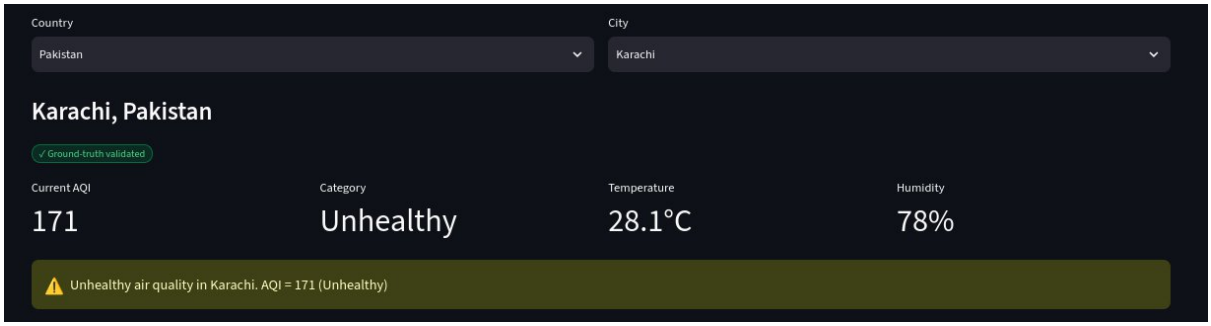
When validated against the 4 ground-truth cities, OpenWeather's underlying

atmospheric model showed substantial, city-specific bias — in some cases estimating AQI as much as ~2.3x higher or lower than the real ground-station reading.

The first approach attempted was a single, global correction factor applied uniformly across all cities, based on the average bias observed across the 4 validated cities. This did not generalize well — when tested against the individual cities, the global factor explained very little of the actual variation, with an R^2 of only 0.14. In other words, the bias was not consistent enough from city to city for one universal number to correct it accurately; a factor that worked for one city often overcorrected or undercorrected another.

Because a single correction factor did not generalize across cities, a per-city ratio-based correction was applied instead — but only to the 4 cities with real validation data available. The remaining 26 cities are deliberately left uncorrected and are clearly labeled as "model estimates" throughout the dashboard, rather than applying an unvalidated correction that could make their accuracy worse instead of better.





5. Model Training & Comparison

The dashboard forecasts AQI day-by-day for the next 3 days. Rather than training a single model to predict 72 hours ahead directly, three separate models were trained — one for each forecast horizon (24h, 48h, and 72h) — since each horizon is a genuinely different prediction problem with its own error characteristics.

5.1 Models Compared

- Ridge Regression — a regularized linear model, used as the baseline.
- RandomForest — an ensemble of decision trees.
- XGBoost — a gradient-boosted tree ensemble.
- A feedforward neural network (TensorFlow/Keras), trained separately on Google Colab, evaluated as a fourth comparison point.

While the neural network performed reasonably well, it did not outperform the tree-based models at the current data volume — a result that is expected rather than surprising. Neural networks typically need substantially larger datasets to learn useful patterns from structured, tabular data like pollutant and weather readings; with a relatively short collection window so far, tree-based ensembles like RandomForest and XGBoost were able to model the relationships in the data more effectively with fewer examples. For this reason, the neural network was kept as a comparison point in the model selection process but was not included in the production forecasting pipeline. As more historical data accumulates over time through the automated data pipeline, revisiting the neural network as a production candidate could be a reasonable next step.

5.2 Results — 24 Hour Horizon

Model	R^2	MAE	RMSE
XGBoost	0.7595	13.35	19.86
RandomForest	0.7517	12.78	20.18
Ridge Regression	0.2007	26.71	36.2

5.3 Results — 48 Hour Horizon

Model	R^2	MAE	RMSE
XGBoost	0.7562	14.02	20.55
RandomForest	0.7417	12.96	21.15
Ridge Regression	0.1746	28.18	37.81

5.4 Results — 72 Hour Horizon

Model	R^2	MAE	RMSE
RandomForest	0.7565	12.6	19.79
XGBoost	0.7469	13.92	20.18
Ridge Regression	0.1907	27.86	36.08

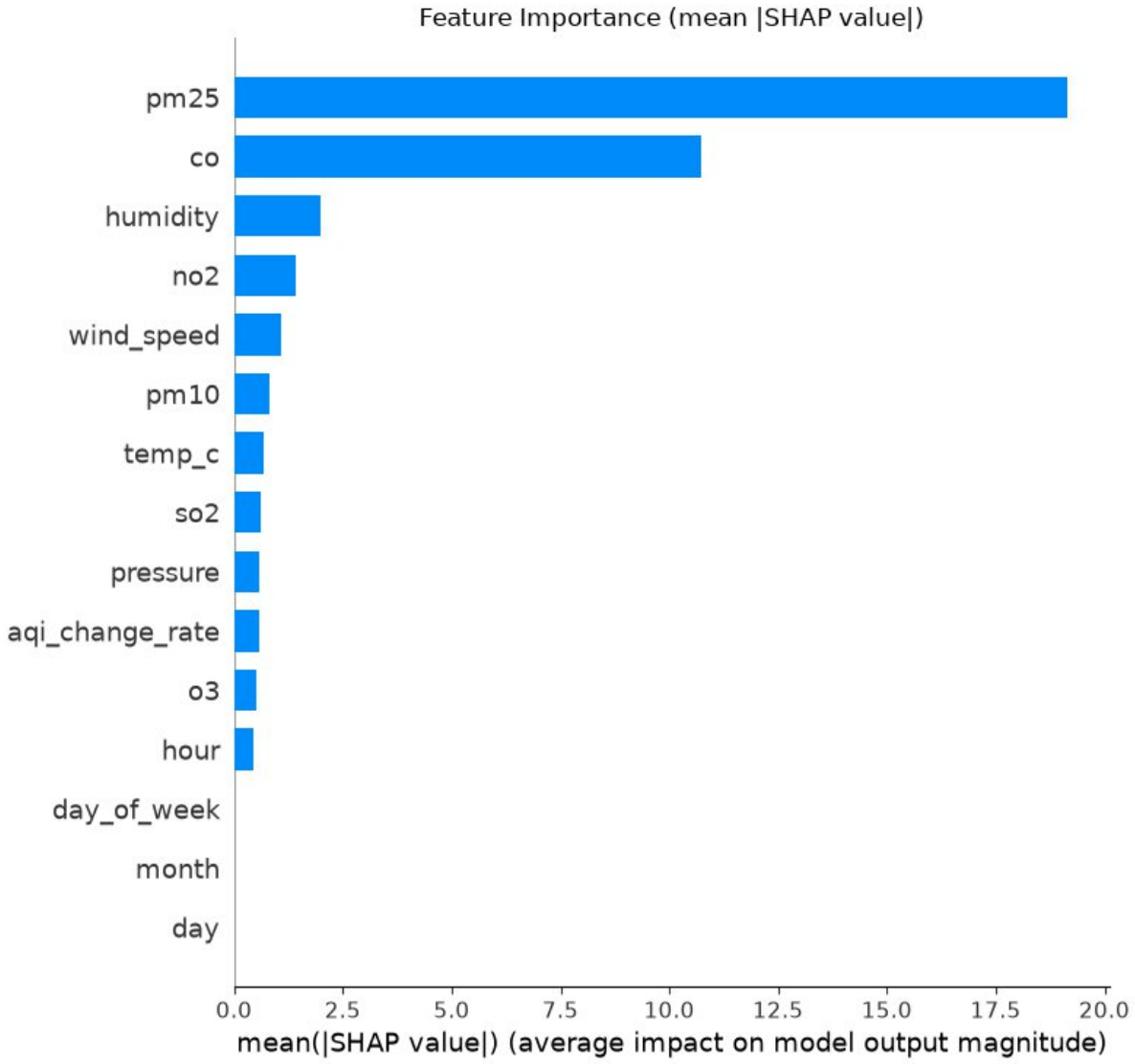
Across all three forecast horizons, the pattern is consistent: RandomForest and XGBoost trade off winning by a small margin depending on the horizon — XGBoost slightly ahead at 24 hours and 48 hours, RandomForest slightly ahead at 72 hours — while Ridge Regression trails substantially behind both at every horizon. The winning tree-based model achieves an R^2 between 0.70 and 0.76 across horizons, compared to only 0.17–0.20 for Ridge. This gap confirms that the relationship between the input features (pollutant levels, weather, time-based features) and future AQI is not well captured by a simple linear model, and that the tree-based models' ability to learn non-linear interactions between features is what drives their stronger performance.

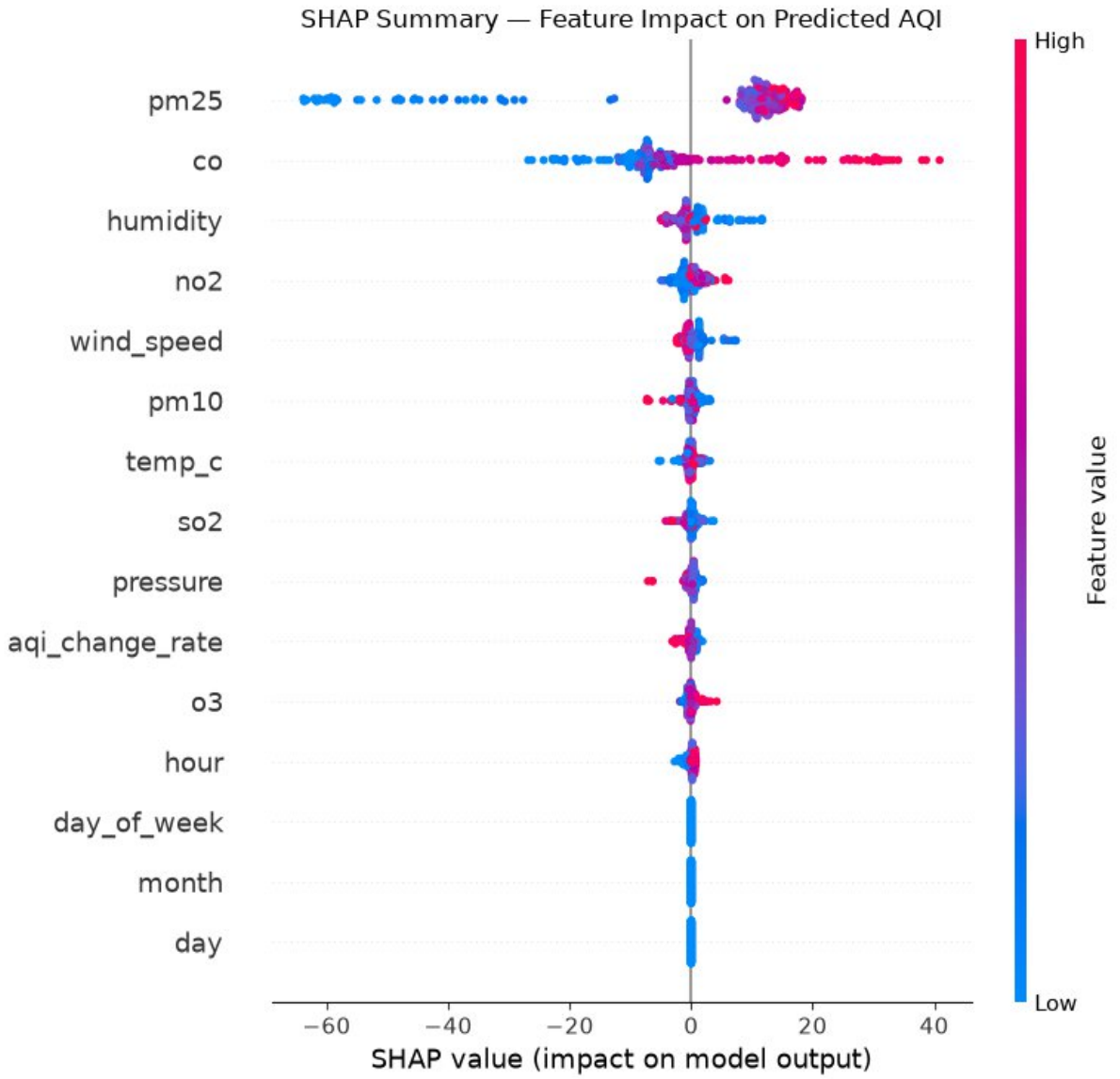
	horizon	model	r2	mae	rmse
0	24	XGBoost	0.7595	13.35	19.86
1	24	RandomForest	0.7517	12.78	20.18
2	24	Ridge Regression	0.2007	26.71	36.2

5.5 Explainability

SHAP (SHapley Additive exPlanations) was used to identify which input features most influence the model's predictions. PM2.5 and CO consistently ranked as the most influential

features across horizons — CO's importance is notable since it is not a direct input to the AQI formula itself, suggesting it acts as a proxy for sustained local pollution intensity.





6. Automation & CI/CD

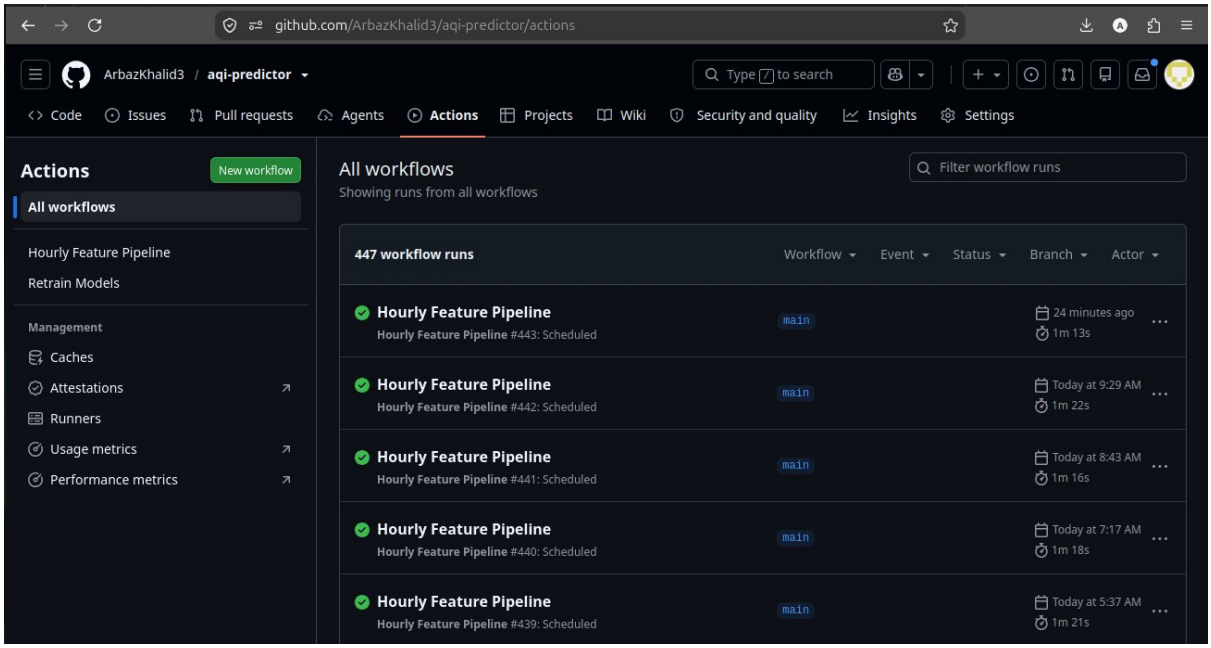
The system is designed to run with zero manual intervention through two independently scheduled GitHub Actions workflows:

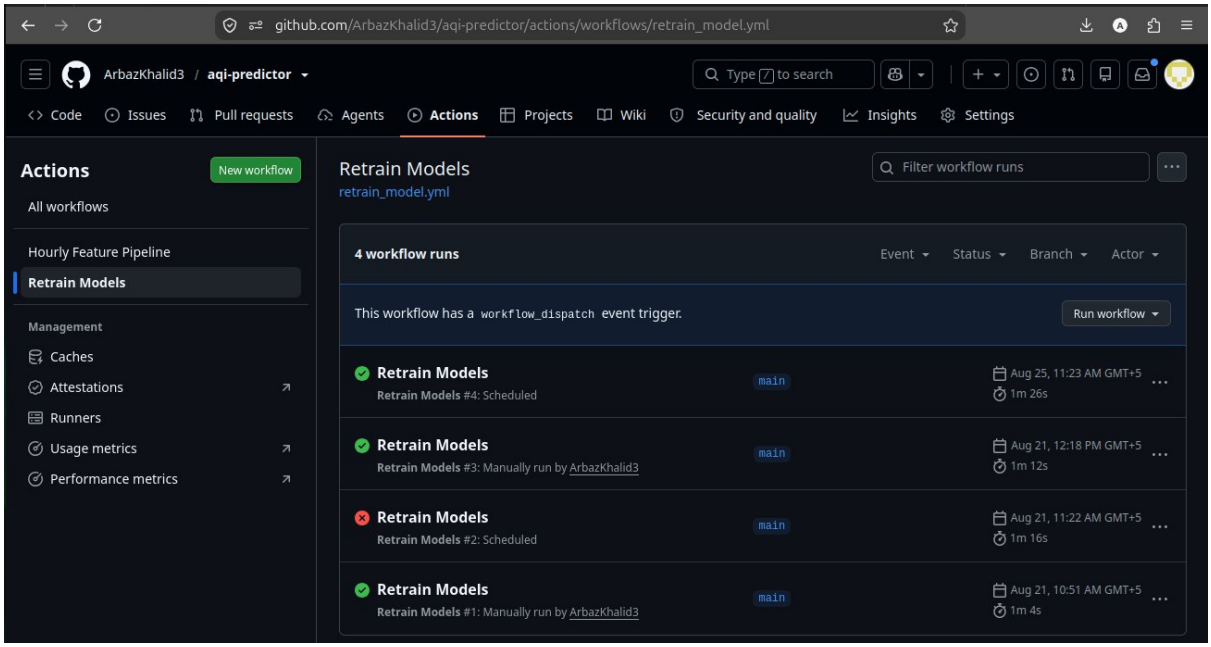
6.1 Hourly Feature Pipeline

Runs every hour, fetches the latest weather and pollutant data for all 30 cities, computes AQI, applies calibration where applicable, and commits the updated data to the repository.

6.2 Automated Retraining Pipeline

Runs every 4 days, retrains all 3 production models (Ridge, RandomForest, XGBoost) across all 3 forecast horizons on the latest accumulated data, evaluates every model, and commits the best-performing model per horizon along with a full metrics comparison — the same table format shown in Section 5.





6.3 Automated Commit Evidence

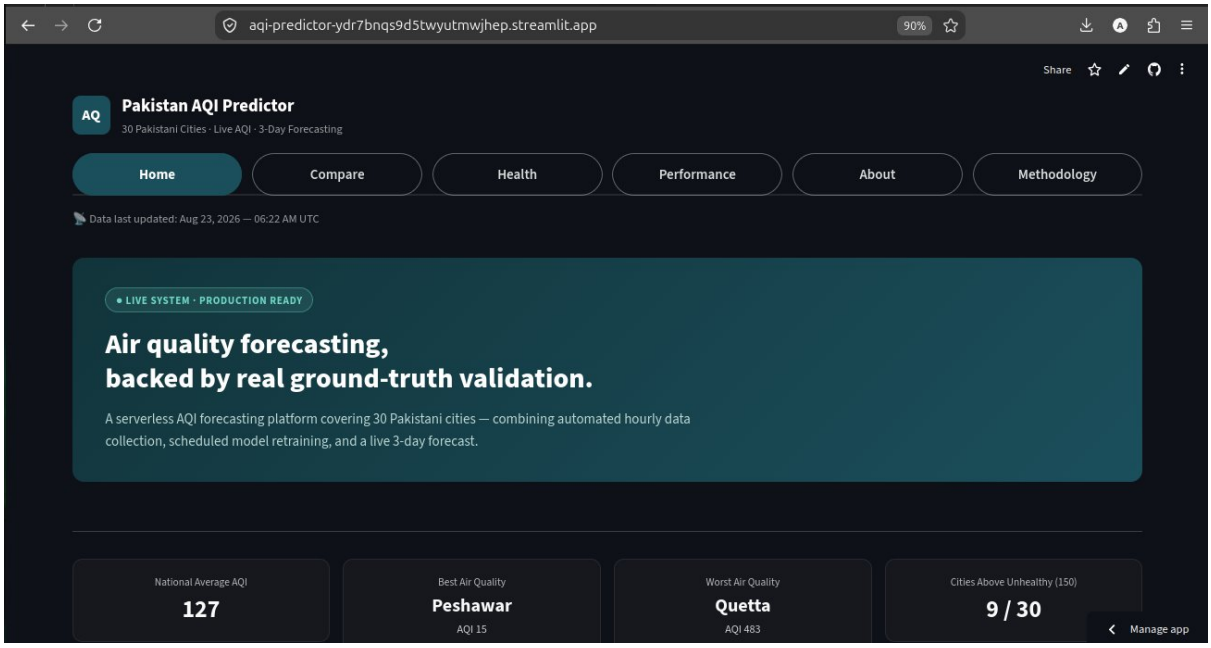
This automation is independently visible in the repository itself: GitHub attributes commits made by the scheduled workflows to a separate "actions-user" contributor, distinct from manual commits. As of this report, actions-user shows dozens of automated commits — hourly feature updates and periodic model retrains — running entirely without manual intervention, providing concrete evidence that the pipeline operates continuously in production rather than being triggered manually.

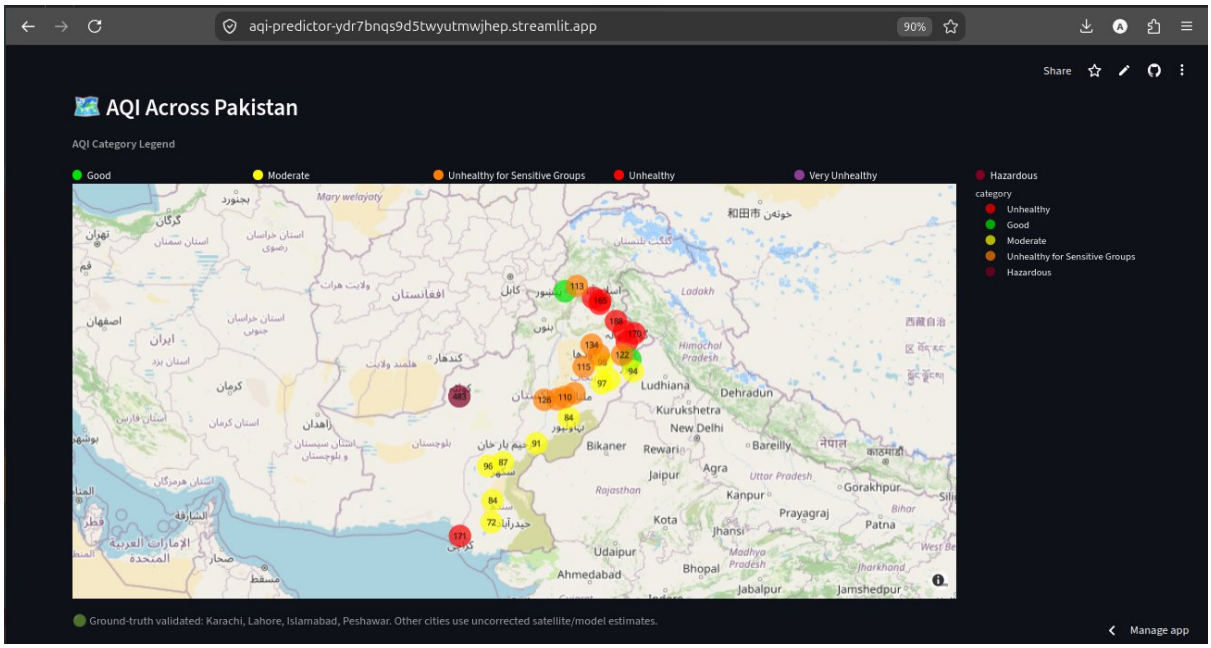
7. Dashboard

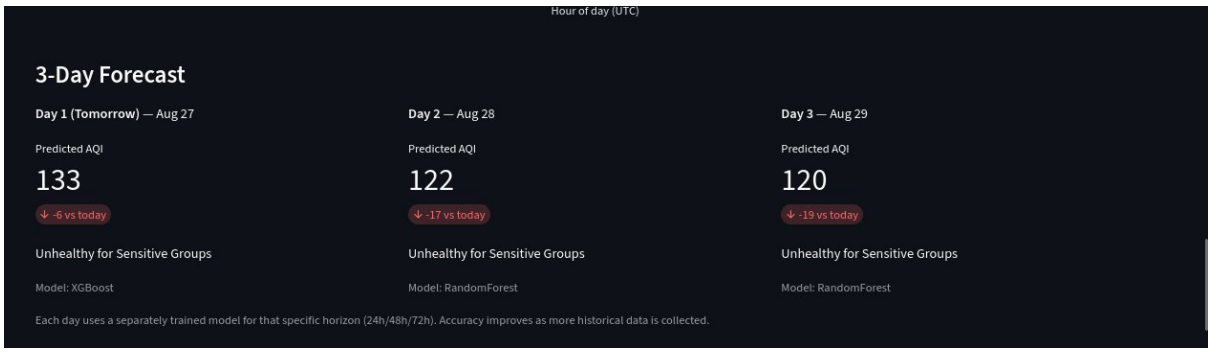
The live dashboard is publicly accessible at:
aqi-predictor-ydr7bnqs9d5twyutmwjhep.streamlit.app

7.1 Home

The Home page provides a national overview of air quality — a summary of the average, best, and worst AQI across all 30 cities, an interactive map showing every city's current AQI, and a full city ranking table. Selecting a specific city reveals its current conditions, a historical AQI trend chart, a 7-day hourly heatmap, a day-by-day 3-day forecast, and a breakdown of individual pollutant concentrations.

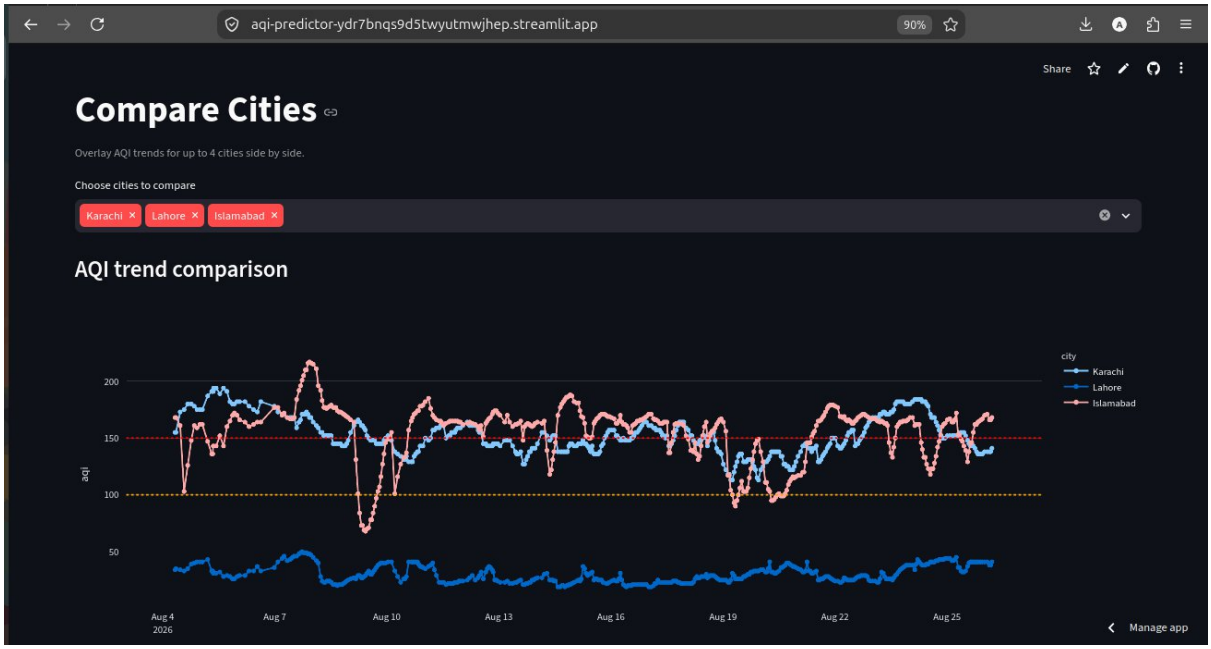

aqi-predictor-ydr7bnqs9d5twyutmwjhep.streamlit.app
90%
Share
AQ
Pakistan AQI Predictor
30 Pakistani Cities · Live AQI · 3-Day Forecasting
Home
Compare
Health
Performance
About
Methodology
Data last updated: Aug 23, 2026 — 06:22 AM UTC
● LIVE SYSTEM · PRODUCTION READY
Air quality forecasting,
backed by real ground-truth validation.
A serverless AQI forecasting platform covering 30 Pakistani cities — combining automated hourly data collection, scheduled model retraining, and a live 3-day forecast.
National Average AQI
127
Best Air Quality
Peshawar
AQI 15
Worst Air Quality
Quetta
AQI 483
Cities Above Unhealthy (150)
9 / 30
Manage app

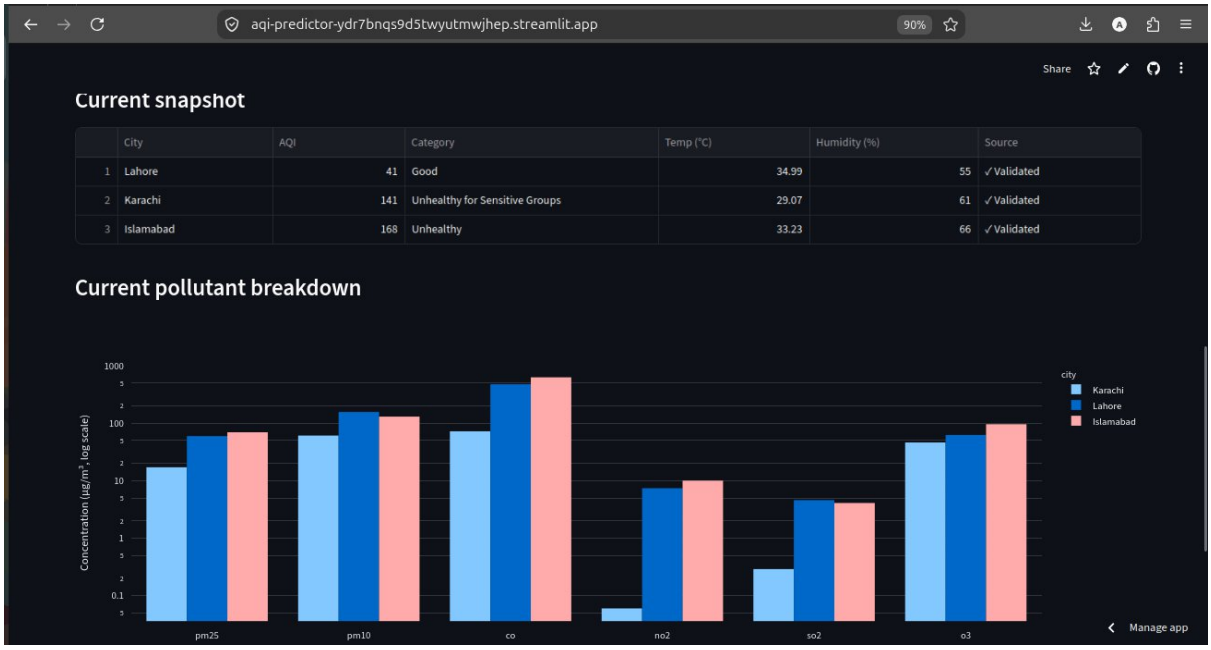

aqi-predictor-ydr7bnqs9d5twyutmwjhep.streamlit.app
90%
Share
AQI Across Pakistan
AQI Category Legend
Good
Moderate
Unhealthy for Sensitive Groups
Unhealthy
Very Unhealthy
Hazardous
category
Unhealthy
Good
Moderate
Unhealthy for Sensitive Groups
Hazardous
Ludhiana
Dehradun
Kurukshetra
New Delhi
Bareilly
Bikaner
Rewari
Jaipur
Agra
Kanpur
Gorakhpur
Prayagraj
Patna
Kota
Jhansi
Udaipur
Ahmedabad
Bhopal
Jabalpur
Jamshedpur
Ground-truth validated: Karachi, Lahore, Islamabad, Peshawar. Other cities use uncorrected satellite/model estimates.
Manage app


Hour of day (UTC)
3-Day Forecast
Day 1 (Tomorrow) — Aug 27
Predicted AQI
133
-6 vs today
Unhealthy for Sensitive Groups
Model: XGBoost
Day 2 — Aug 28
Predicted AQI
122
-17 vs today
Unhealthy for Sensitive Groups
Model: RandomForest
Day 3 — Aug 29
Predicted AQI
120
-19 vs today
Unhealthy for Sensitive Groups
Model: RandomForest
Each day uses a separately trained model for that specific horizon (24h/48h/72h). Accuracy improves as more historical data is collected.

7.2 Compare Cities

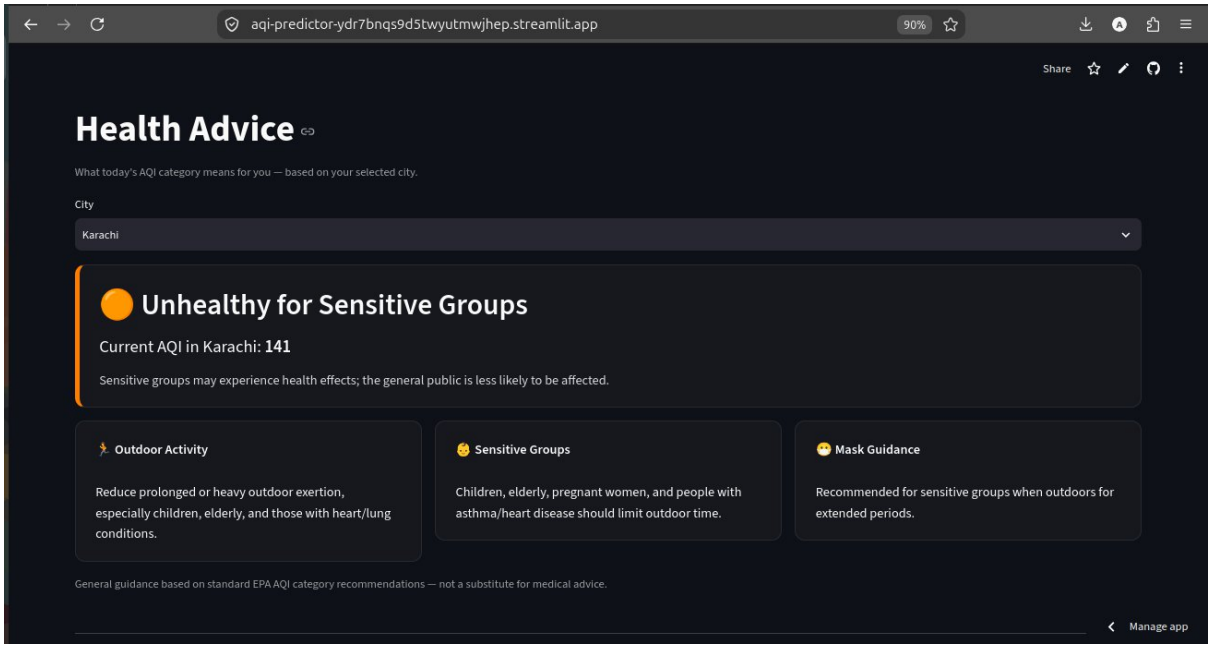
The Compare Cities page allows users to select up to four cities and view their AQI trends overlaid on a single chart, alongside a side-by-side snapshot of current conditions and a pollutant-level comparison — making it easy to see how air quality differs across regions at a glance.





7.3 Health Advice

The Health Advice page translates a selected city's current AQI category into practical guidance — covering outdoor activity, precautions for sensitive groups (children, the elderly, and those with respiratory or heart conditions), and mask recommendations — along with a reference guide to what each of the six AQI categories means.

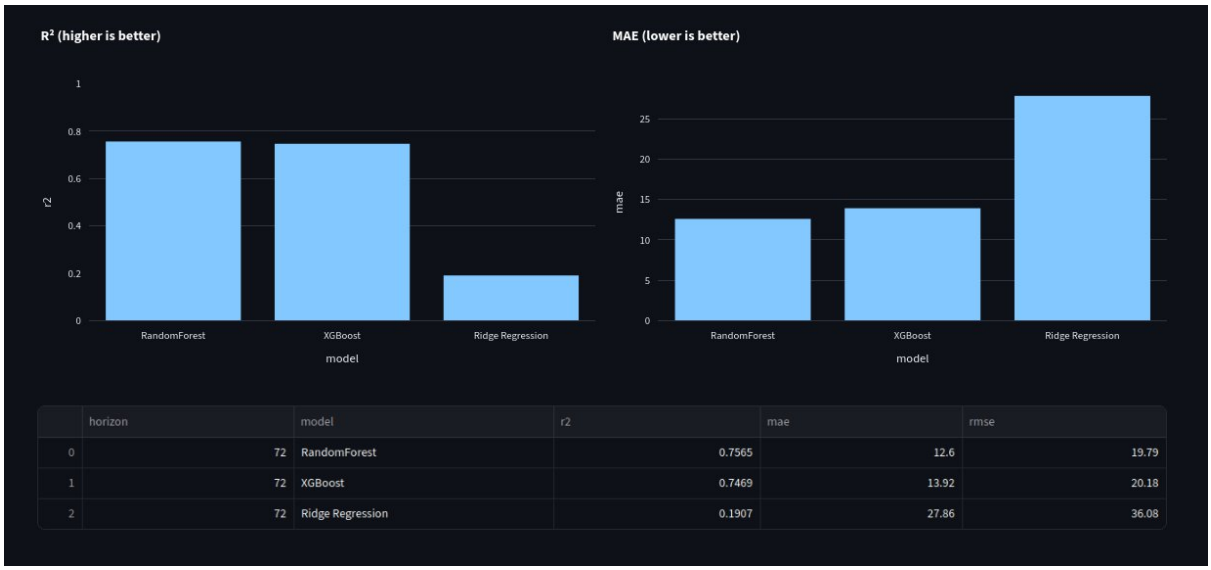


7.4 Model Performance

The Model Performance page presents the same R^2, MAE, and RMSE comparison across all three models and forecast horizons discussed in Section 5, updated automatically each time the retraining pipeline runs — allowing model accuracy to be monitored over time as more historical data accumulates.

	horizon	model	r2	mae	rmse
0	24	XGBoost	0.7595	13.35	19.86
1	24	RandomForest	0.7517	12.78	20.18
2	24	Ridge Regression	0.2007	26.71	36.2

	horizon	model	r2	mae	rmse
0	48	XGBoost	0.7562	14.02	20.55
1	48	RandomForest	0.7417	12.96	21.15
2	48	Ridge Regression	0.1746	28.18	37.81



	horizon	model	r2	mae	rmse
0	72	RandomForest	0.7565	12.6	19.79
1	72	XGBoost	0.7469	13.92	20.18
2	72	Ridge Regression	0.1907	27.86	36.08

8. Challenges & Learnings

Building and deploying this system surfaced several real challenges beyond model training itself, particularly around getting a working pipeline live in production. The biggest lesson was the gap between training a model in a notebook and running one in production: a notebook only needs to work once, but a production pipeline needs to keep working unattended, on a schedule, across changing data.

Some specific moments from the process:

- **Deployment failure — missing dependency:** The first Streamlit Cloud deployment failed immediately with ModuleNotFoundError: xgboost — the model file had been trained with a package never added to requirements.txt.
- **Follow-up dtype bug:** Fixing that surfaced a second issue — a ValueError inside XGBoost's prediction step, traced back to the single-row feature dataframe being built with an inconsistent dtype (object instead of float). Casting the features to float before prediction resolved it.
- **Silent .gitignore conflict:** While wiring the automated retraining pipeline to commit model performance metrics, a .gitignore rule (data/*.csv) was silently excluding model_metrics.csv from version control, even though the script was generating it correctly — a reminder that a successful script run doesn't guarantee its output actually reaches the repository.
- **Iterative dashboard design:** The dashboard went through several rounds of design iteration — adjusting navigation sizing, reorganizing the layout so city-specific details appeared before the national map, and later expanding a single-page dashboard into six focused pages (Home, Compare, Health, Performance, About, Methodology) based on reviewing what the initial version was missing.
- **What I'd do differently:** If starting over, I would set up the full CI/CD pipeline and dashboard skeleton earlier in the project, rather than building the model first and adding production infrastructure around it afterward — dependency management, defensive error handling, and version control discipline turned out to matter just as much as model accuracy.
- **Automated retraining race condition:** The first scheduled run of the retraining pipeline failed — it retrained the models successfully, but the git push was rejected because the hourly pipeline had pushed a newer commit in the meantime. This was fixed by adding a git pull --rebase step before the push, making the workflow resilient to running alongside the other automated pipeline.

9. Tech Stack Deep-Dive

TOOL	Role in the Project
Python	Core language for the entire pipeline
Scikit-learn	RandomForest and Ridge Regression models
XGBoost	Gradient-boosted tree model
TensorFlow/Keras	Neural network comparison (colab only)

Hopsworks	Cloud feature store
GitHub Actions	CI/CD — hourly data collection + automated retraining
Streamlit	Interactive dashboard framework
Plotly	Charts and the interactive map
SHAP	Model explainability
joblib	Model serialization

Python was the natural choice as the single language across the entire pipeline, since every other tool used here has first-class Python support, avoiding the need to bridge multiple languages between data collection, training, and serving.

scikit-learn and XGBoost were chosen together deliberately, to compare a simpler ensemble method (RandomForest) against a more sophisticated gradient-boosted approach (XGBoost) — both are well-suited to tabular, structured data like pollutant and weather readings, unlike deep learning approaches which typically need larger datasets. TensorFlow/Keras was used only for the one-time neural network comparison, since it was not the production choice, running it purely in a Colab notebook avoided adding an unnecessary dependency to the automated pipeline.

Hopsworks was selected specifically because the project brief called for a managed feature store, rather than just a flat file — it provides structured feature versioning and querying beyond what a plain CSV offers, while the CSV itself was kept as a lightweight, always-available fallback that the dashboard and training scripts depend on directly.

GitHub Actions was chosen for automation because it requires no separate infrastructure to manage — both the hourly data collection and the periodic retraining run entirely within the same GitHub repository already hosting the code, with no servers to provision or maintain.

Streamlit and Plotly were chosen together for the dashboard because they allow a fully interactive, chart-heavy interface to be built quickly in pure Python, without needing separate frontend web development skills or a JavaScript framework.

SHAP was used for explainability because it provides a model-agnostic, theoretically grounded way to attribute a prediction to its input features — important for understanding not just how accurate the models are, but why they produce the predictions they do.

joblib was used to serialize trained models to disk, since it is the standard, efficient serialization method for scikit-learn and XGBoost models specifically, and integrates directly with how these models are loaded back into the dashboard for inference.

10. Known Limitations

- **Limited ground-truth coverage** — only 4 of 30 cities have real validation data; the other 26 rely on uncorrected model estimates, clearly labeled as such on the dashboard.
- **Neural network not in production** — it was a one-time Colab comparison, not part of the automated retraining pipeline.
- **Short data collection window** — model accuracy is expected to improve as more historical data accumulates over time.
- **Hopsworks gaps during development** — the feature store connection was temporarily paused to stay within free-tier limits; the local CSV pipeline continued uninterrupted throughout.
- **No automated testing** — the pipeline relies on manual verification (dashboard checks, GitHub Actions logs) rather than automated unit/integration tests.
- **Single data source per city** — no fallback source or interpolation if the upstream API has an outage or returns bad data for a given hour.
- **No authentication or rate limiting** — the dashboard is fully public, appropriate for a portfolio project but not production-scale use.
- **Fixed retraining cadence** — models retrain every 4 days on a schedule, not in response to detected accuracy drift.
- **Fixed forecast horizons** — only 24h/48h/72h are supported; other windows would require training additional horizon-specific models.

11. Conclusion

This project delivered a complete, serverless AQI forecasting system for Pakistan — from automated hourly data collection and ground-truth calibration, through multi-model training and comparison, to a fully automated retraining pipeline and a public, interactive dashboard. Beyond the machine learning itself, the project was a practical lesson in what it takes to run a system in production rather than a notebook: dependency management, defensive error handling, and version control discipline proved just as important as model accuracy. With more time, the system could be extended with additional ground-truth validation cities, a longer historical dataset to improve model accuracy further, and a mobile-friendly version of the dashboard.

12. References & Links

- Live Dashboard: https://aqi-predictor-ydr7bnqs9d5twyutmwjhep.streamlit.app
- Source Code: https://github.com/ArbazKhalid3/aqi-predictor
- OpenWeather Air Pollution API: https://openweathermap.org/api/air-pollution
- AQICN: https://aqicn.org
- US EPA AQI Technical Assistance Document (breakpoint formula)